

Reviewer Pack — Minimal Rank of Tensor Product of Tropicalized Cohomology wi...

Entry d1ac0509b45b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 08:44:34 UTC

Minimal Rank of Tensor Product of Tropicalized Cohomology with Resolution Proof Tree Width

Entry ID: d1ac0509b45b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 08:44:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Tropicalization) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

[‘For every Boolean function f , the minimal rank of its tropicalized cohomology group $H^0(T_f)$ is at least as large as the resolution proof tree width $w(f)$. Specifically, for all instances of size n , we have $r(H^0(T_f)) \geq w(f)$, where $r(H^0(T_f))$ denotes the minimal rank of $H^0(T_f)$.’, ‘For any instance with a resolution proof tree width less than its tropicalized cohomology group’s minimal rank, there exists an alternative Boolean function g with a smaller resolution proof tree width but an equal or larger minimal rank of its tropicalized cohomology group.’, ‘This conjecture is falsified if we can find an instance where the minimal rank of the tropicalized cohomology group is significantly less than the resolution proof tree width, indicating that the minimal rank does not serve as a reliable lower bound for the resolution proof tree width.’]

Rationale (proposer’s reasoning):

[‘The connection between algebraic geometry and complexity theory through tropicalization offers a unique lens to study computational problems. Tropicalized cohomology has been used in real algebraic geometry, and its properties could potentially provide insights into the structure of resolution proofs, which are fundamental in complexity theory.’, ‘The proposed conjecture aims to establish a quantitative relation between these two fields, which could reveal new structural insights and potentially lead to improved lower bounds for computational problems.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d04818fed7a9aa5d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture on minimal rank of tropicalized cohomology and resolution proof tree width, support is indicated if $r(H^0(T_f)) \geq w(f)$ for all seeds $n \leq 40$ with a standard deviation (SD) ≤ 2 across all seeds. Falsification occurs if any seed has $r(H^0(T_f)) < w(f) - SD$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0

    for r in range(rows):
        if matrix[r][rank] == 0:
            swap_found = False
            for i in range(r + 1, rows):
                if matrix[i][rank] != 0:
                    matrix[r], matrix[i] = matrix[i], matrix[r]
                    swap_found = True
                    break
            if not swap_found:
                continue

        pivot = Fraction(matrix[r][rank])

        for c in range(cols):
            matrix[r][c] /= pivot

        for i in range(rows):
            if i != r and matrix[i][rank] != 0:
                factor = -matrix[i][rank]
                for c in range(cols):
```

```

        matrix[i][c] += factor * matrix[r][c]

    rank += 1
    if rank == cols:
        break

    return rank

def minimal_rank(boolean_function):
    n = len(boolean_function)
    augmented_matrix = [[0] * (n + 1) for _ in range(n)]

    for i in range(n):
        for j in range(n):
            augmented_matrix[i][j] = boolean_function[i][j]
        augmented_matrix[i][n] = boolean_function[i][-1]

    return gaussian_elimination(augmented_matrix)

def resolution_width(boolean_function):
    n = len(boolean_function)
    clauses = [set clause for clause in boolean_function if isinstance(clause, list)]
    variables = set.union(*clauses)

    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause.pop()
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if dpll([c - {literal} for c in clauses if literal not in c], new_assignment):
                return True
            new_assignment[literal] = False
            if dpll([c - {-literal} for c in clauses if -literal not in c], new_assignment):
                return True
        else:
            literal = next(iter(variables))
            if dpll(clauses, assignment | {literal: True}):
                return True
            if dpll(clauses, assignment | {literal: False}):
                return True
        return False

    max_width = 0
    for assignment in itertools.product([False, True], repeat=n):
        width = sum(1 for var in variables if assignment[var])
        if not dpll(clauses, assignment):
            max_width = max(max_width, width)

    return max_width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)

```

```

boolean_function = [[random.choice([True, False]) for _ in range(n)] + [random.choice([True, False]) for _ in range(n)]]

rank = minimal_rank(boolean_function)
width = resolution_width(boolean_function)

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank >= width,
    "counterexample": "" if rank >= width else f"rank={rank}, width={width}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank < width\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3373a696.py", line 117, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3373a696.py", line 101, in run_trial
    width = resolution_width(boolean_function)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3373a696.py", line 90, in resolution_width
    if not dpll(clauses, assignment):
            ^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3373a696.py", line 81, in dpll
    if dpll(clauses, assignment | {literal: True}):
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

TypeError: unsupported operand type(s) for |: 'tuple' and 'dict'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with appropriate error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15742
2	preregistration	ollama_remote	glm4:latest	0	0	10047
3	novelty	ollama_remote	glm4:latest	0	0	9467
4	novelty	ollama_remote	glm4:latest	0	0	8884
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16376
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11898
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13305
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12152
9	judge	ollama_remote	glm4:latest	0	0	16483

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 114354 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/d1ac0509b45b.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/d1ac0509b45b.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/d1ac0509b45b.tar.gz* (if generated)